



D11 — Guia de Configuracao do Antigravity para o Aura

Como configurar o Google AI IDE para desenvolver
o projeto Aura de forma autonoma
Regras, contexto, prompts e workflow completo

Projeto Aura — Plataforma de Gestao de Eventos

Documento 11 — Specdrive Tecnico

1. O que é o Antigravity (Google AI IDE)

O **Antigravity** é uma IDE (ambiente de desenvolvimento integrado) desenvolvida pelo Google que usa inteligência artificial como centro do workflow. Diferente de editores tradicionais onde você escreve todo o código, no Antigravity você descreve o que quer em linguagem natural e a IA gera o código.

1.1 Como Funciona

1. **Voce descreve:** "Crie a tabela de usuarios no Supabase com campos para nome, email, CPF e telefone"
2. **A IA entende:** O Antigravity lê o contexto do projeto (regras, arquivos existentes, stack) e interpreta seu pedido
3. **A IA executa:** Gera o arquivo SQL, seguindo os padrões que você definiu nas regras
4. **Voce revisa:** Aprova, pede modificações ou rejeita

1.2 O que o Antigravity Precisa para Funcionar Bem

Para que o Antigravity desenvolva o Aura corretamente, ele precisa de três coisas:

Tabela 1 Requisitos para o Antigravity Trabalhar no Aura

Item	Descrição	Onde Configura
Regras do Projeto	Padrões de código, stack, estrutura de pastas, o que pode e o que não pode fazer	Arquivo <code>.antigravity/rules.md</code>
Contexto do Projeto	O que já existe, o que falta fazer, arquitetura, decisões tomadas	Documentos D1 a D10 (specdrive)
Tarefas Bem Definidas	Instruções claras do que fazer, na ordem correta	Prompts estruturados (Seção 6)

2. Configuracao Inicial (Passo a Passo)

2.1 Instalar o Antigravity

Passo 1: Acesse `antigravity.dev` e faça login com sua conta Google.

Passo 2: Na tela inicial, clique em "New Project" ou "Import Project".

Passo 3: Selecione a opção "Import from GitHub" e conecte seu repositório do Aura (ou faça upload da pasta do projeto).

2.2 Criar o Arquivo de Regras

Passo 4: Na raiz do projeto, crie a pasta `.antigravity` e dentro dela o arquivo `rules.md`. Este arquivo é a bíblia do projeto — tudo que a IA precisa saber sobre como codificar.

```
.antigravity/rules.md
```

2. Configuracao Inicial (Passo a Passo)

```
# REGRAS DO PROJETO AURA # ===== # Este arquivo define como o
Antigravity deve se comportar # ao desenvolver o projeto Aura. NAO IGNORE ESTAS REGRAS.
## IDENTIDADE DO PROJETO - Nome: Aura - Plataforma de Gestao de Eventos - Tipo: SaaS de
ticketing e gestao de eventos - Publico: Produtores de eventos e participantes - Regiao:
Brasil (PT-BR, moeda BRL, fuso BRT) ## STACK TECNOLÓGICA - Frontend: React 19 +
TypeScript + Vite - Estilos: Tailwind CSS + shadcn/ui - Backend: Supabase (PostgreSQL,
Auth, Storage, Realtime, Edge Functions) - Estado Client: Zustand - Estado Server:
TanStack Query (React Query) - Email: Resend - Pagamentos: A DEFINIR (gateway-agnostic) -
Hospedagem: Vercel - Dominio: Hostinger ## ESTRUTURA DE PASTAS OBRIGATORIA `` src/ lib/
supabase.ts # Cliente Supabase (JA EXISTE) stores/ authStore.ts # Zustand store de
autenticacao eventStore.ts # Zustand store de eventos cartStore.ts # Zustand store de
carrinho uiStore.ts # Zustand store de UI hooks/ useAuth.ts # Hook de autenticacao
useEvents.ts # Hook de eventos useTickets.ts # Hook de ingressos useOrders.ts # Hook de
pedidos usePayments.ts # Hook de pagamentos (gateway-agnostic) useCheckin.ts # Hook de
check-in useReports.ts # Hook de relatorios useNotifications.ts # Hook de notificacoes
types/ database.ts # Tipos do Supabase payments.ts # Tipos de pagamento api/
paymentAdapter.ts # Adapter de pagamento (gateway-agnostic) pages/ # Paginas existentes
(66 paginas, NAO MOVER) components/ # Componentes existentes (82 componentes) contexts/
data/ mockData.ts # Dados mock (SUBSTITUIR POR API REAL) `` ## REGRAS DE CODIGO ###
TypeScript - SEMPRE use TypeScript. NUNCA use JavaScript puro. - SEMPRE defina interfaces
para props de componentes. - SEMPRE use tipos do Supabase (Database types) para queries.
- NUNCA use "any" sem justificativa documentada. ### Componentes React - Use functional
components com hooks. - Use shadcn/ui para componentes base (Button, Input, Dialog, etc).
- Componentes de pagamento DEVEM ser gateway-agnostic. - Use forwardRef quando necessario
para inputs. - SEMPRE trate estados de loading e erro. ### Supabase - NUNCA exponha a
SERVICE_ROLE_KEY no frontend. - SEMPRE use RLS policias para seguranca de dados. - Use
Edge Functions para logica sensivel (pagamentos, webhooks). - Use Realtime para
atualizacoes em tempo real (check-in, status). ### Zustand - Um store por dominio (auth,
event, cart, ui). - Use persist middleware apenas para auth e cart. - NUNCA armazene
dados sensiveis (senhas, CVV) em stores. ### TanStack Query - Use para TODOS os dados
vindos do servidor. - SEMPRE defina staleTime e cacheTime apropriados. - Use invalidation
para atualizar dados apos mutacoes. - Use isPending para estados de loading. ### Tailwind
CSS - Use classes utilitarias. NUNCA crie CSS custom exceto quando necessario. - Use o
sistema de cores do shadcn/ui (primary, secondary, etc). - Componentes devem ser
responsivos (mobile-first). ### Seguranca - Valide TODOS os inputs no frontend ANTES de
enviar. - Escape TODOS os dados que vem do usuario antes de exibir. - NUNCA armazene
tokens ou chaves de API no codigo fonte. - Use variaveis de ambiente (VITE_*) para
configuracoes. ### Performance - Use lazy loading para paginas (React.lazy + Suspense). -
Otimize imagens antes de enviar para Storage. - Use debounce para inputs de busca. -
Limite queries a 100 registros por pagina (paginacao). ## O QUE NAO FAZER - NAO crie
novas paginas sem consultar o specdrive (D1). - NAO modifique componentes existentes sem
entender o contexto. - NAO implemente logica de gateway de pagamento (aguardar escolha).
- NAO crie tabelas no Supabase sem seguir o schema do D5. - NAO ignore RLS policias em
NENHUMA tabela. - NAO use bibliotecas sem aprovar (consultar antes). ## PRIORIDADES DE
IMPLEMENTACAO 1. Auth (login/registro/perfil) - CRITICO 2. Listagem de eventos + detalhe
- CRITICO 3. Carrinho + checkout visual - CRITICO 4. Painel do produtor - ALTO 5. Check-
in com QR code - ALTO 6. Dashboard financeiro - MEDIO 7. White label - MEDIO (Fase 2) 8.
POS Cashless - NAO IMPLEMENTAR (Fase 3 futura) ## DECISOES JA TOMADAS - Gateway de
pagamento: AINDA NAO DEFINIDO (use adapter generico) - Taxa da plataforma: 5-7% sobre
transacoes - Plano white label: R199 - R 999/mes - KYC obrigatorio para todos os
produtores - Limites progressivos para produtores novos (R$ 5k primeiro mes) - POS
Cashless: ADIADO para Fase 3 (apos 50+ eventos online) ## COMUNICACAO - Responda sempre
em portugues (PT-BR). - Explique o que esta fazendo antes de fazer. - Se encontrar um
```

2.3 Criar o Arquivo de Contexto do Projeto

Passo 5: Na pasta `.antigravity`, crie o arquivo `context.md`. Este arquivo resume o projeto para que a IA não precise adivinhar nada.

```
.antigravity/context.md
```

```
# CONTEXTO DO PROJETO AURA # ===== # Resumo completo para o
Antigravity entender o projeto ## ESTADO ATUAL O Aura é uma plataforma de gestão de
eventos (ticketing) com frontend 80% pronto (React + TypeScript + Tailwind + shadcn/ui).
Tem 66 páginas e 82 componentes. Todos os dados são mockados (falsos). O backend
(Supabase) ainda não foi criado. ## O QUE JÁ EXISTE (FEITO) - 66 páginas React com
roteamento (HashRouter) - 82 componentes shadcn/ui configurados - Seating Map com Canvas
HTML5 (editor completo) - Certificate Builder com 12 templates - UI de checkout (visual,
sem funcionalidade real) - UI de painel do produtor (visual, sem dados reais) - UI de
check-in (visual, sem câmera real) - Layout responsivo, tema dark/light - Componentes:
header, footer, sidebar, modais, tabelas, formulários ## O QUE FALTA FAZER (PRIORIDADE)
1. CRIAR o banco de dados no Supabase (30 tabelas, D5) 2. CONFIGURAR autenticação
(Supabase Auth) 3. CONFIGURAR RLS políticas (50+ políticas, D5) 4. CRIAR Edge Functions (10
funções, D5) 5. SUBSTITUIR mocks por chamadas reais ao Supabase 6. IMPLEMENTAR checkout
funcional (gateway a definir) 7. IMPLEMENTAR check-in com câmera 8. IMPLEMENTAR dashboard
com gráficos reais 9. CONFIGURAR webhooks de pagamento 10. CONFIGURAR envio de emails
(Resend) ## DOCUMENTOS DE REFERENCIA Os seguintes documentos contêm as especificações
completas: - D1: Inventário de todos os arquivos frontend (130+ arquivos) - D2: Gap
analysis - o que falta fazer - D3: Análise estratégica e decisões de negócio - D4:
Melhorias no frontend - D5: Especificação técnica completa do Supabase (SQL, RLS,
triggers) - D6: Arquitetura de sistemas (infraestrutura) - D7: Organização de agentes de
IA - D8: Guia de implementação com código real - D9: Análise de riscos e mitigações -
D10: Componentes de pagamento (gateway-agnóstico) - D11: Este documento (configuração do
Antigravity) ## PROXIMA TAREFA (SEMPRE ATUALIZE ESTA SEÇÃO) [INICIAL] Criar o schema do
Supabase (30 tabelas) seguindo D5
```

3. Como Usar os Documentos no Antigravity

3.1 Upload dos Documentos como Contexto

O Antigravity permite anexar documentos como contexto para a IA ler. Faça isso com os documentos D1 a D10:

Passo 6: No painel lateral do Antigravity, procure a seção "Context" ou "Knowledge" e clique em "Add Files".

Passo 7: Faça upload dos 10 documentos PDF (D1 a D10) e destaque os mais importantes:

- **Sempre visível:** D5 (Supabase Spec), D8 (Codigo real), D10 (Pagamentos)
- **Contexto geral:** D1, D2, D3, D9
- **Consulta quando necessario:** D4, D6, D7

3.2 Ordem de Uso dos Documentos

Tabela 2 Ordem Recomendada de Execucao no Antigravity

Etapa	Documento Principal	O que pedir ao Antigravity	Tempo Est.
1	D5	"Crie as 30 tabelas do Supabase seguindo D5"	2-3 dias
2	D5	"Crie as RLS policies e triggers do D5"	1-2 dias
3	D8	"Configure o cliente Supabase (src/lib/supabase.ts)"	2-3 horas
4	D8	"Crie os Zustand stores (auth, event, cart, ui)"	4-6 horas
5	D8	"Crie os TanStack Query hooks"	6-8 horas
6	D1 + D2	"Substitua os mocks das paginas de evento por dados reais"	2-3 dias
7	D10	"Crie os componentes de pagamento (gateway-agnostic)"	3-4 dias
8	D5	"Crie as Edge Functions de pagamento (process-payment, webhooks)"	2-3 dias
9	D8	"Configure o deploy na Vercel"	2-3 horas
10	Todos	"Teste o fluxo completo e corrija bugs"	3-5 dias

4. Regras Avancadas para o Antigravity

4.1 Arquivo de Conventions

```
.antigravity/conventions.md
```

```
# CONVENCOES DE CODIGO - AURA # ===== # Padroes que TODO codigo
gerado deve seguir ## NOMENCLATURA - Componentes React: PascalCase (EventCard.tsx) -
Hooks: camelCase com prefixo "use" (useEvents.ts) - Stores: camelCase com suffixo "Store"
(authStore.ts) - Utilitarios: camelCase (formatCurrency.ts) - Tipos/Interfaces:
PascalCase com suffixo opcional (EventData) - Constantes: UPPER_SNAKE_CASE
(MAX_TICKETS_PER_ORDER) - Variaveis de ambiente: VITE_ prefixo obrigatorio ## ORGANIZACAO
DE IMPORTS 1. React / frameworks 2. Bibliotecas externas (@tanstack, zustand, etc) 3.
Componentes internos (@/components/...) 4. Hooks internos (@/hooks/...) 5. Stores
internos (@/stores/...) 6. Tipos internos (@/types/...) 7. Utilitarios internos
(@/lib/...) 8. Estilos / CSS ## FORMATO DE COMMITS feat: descricao curta (nova
funcionalidade) fix: descricao curva (correcao de bug) refactor: descricao
(reestruturacao de codigo) chore: descricao (tarefas de manutencao) docs: descricao
(documentacao) ## ESTADO DE ERRO PADRAO Todo componente que faz fetch DEVE ter: ``tsx if
(isPending) return <LoadingSpinner /> if (error) return <ErrorMessage error={error}
onRetry={refetch} /> `` ## FORMULARIOS - Use react-hook-form para formularios complexos
- Valide com Zod schemas - SEMPRE mostre erros de validacao por campo - Desabilite o
botao de submit durante envio ## COMPONENTES SHADCN/UI - Nunca modifique o codigo fonte
do shadcn (em components/ui/) - Estenda criando wrappers quando necessario - Use as
variantes (default, destructive, outline, etc) ## TESTES - Teste hooks com @testing-
library/react-hooks - Teste componentes com @testing-library/react - Mock Supabase em
testes (nao faca chamadas reais)
```

4.2 Arquivo de Seguranca

`.antigravity/security.md`

```
# REGRAS DE SEGURANCA - AURA # ===== # REGRAS INQUEBRABEL. NENHUMA
EXCECAO. ## SUPABASE RLS - TODAS as tabelas DEVEM ter RLS habilitado. - NENHUMA tabela
deve permitir SELECT/INSERT/UPDATE/DELETE anonimo sem policy explicita. - Policies devem
usar auth.uid() para identificar usuarios. - NUNCA use "USING (true)" em policies
(permite tudo). ## SENHAS E CHAVES - NUNCA commite chaves de API no Git. - NUNCA exponha
VITE_SUPABASE_ANON_KEY em logs. - SERVICE_ROLE_KEY e somente para Edge Functions. - Use
.env.local para variaveis sensiveis. ## PAGAMENTOS - NUNCA processe pagamento no
frontend. - SEMPRE valide webhooks por assinatura. - NUNCA confie no valor enviado pelo
cliente. - SEMPRE verifique o valor no servidor (Edge Function). ## INPUTS - SEMPRE
sanitize inputs de usuarios (DOMPurify para HTML). - SEMPRE escape dados antes de exibir.
- Limite uploads a 5MB. - Verifique tipo MIME de uploads. ## AUTENTICACAO - Sessoes
expiram em 24h. - Renove token automaticamente. - Deslogue em caso de erro 401. - Use MFA
para contas admin.
```


5. Prompts Prontos para Copiar e Colar

Estes prompts foram testados e otimizados para o Antigravity entender exatamente o que fazer. Copie e cole no chat do Antigravity.

5.1 Prompt Inicial (Primeiro Uso)

PROMPT 1 - Configuracao Inicial:

Este e o projeto Aura, uma plataforma de gestao de eventos (ticketing).
O frontend esta 80% pronto com React 19 + TypeScript + Tailwind + shadcn/ui.
Preciso que voce configure o backend no Supabase.

Siga as regras em `.antigravity/rules.md` e `.antigravity/conventions.md`.
A especificacao completa esta no documento D5 (Supabase Spec).

Comece criando o schema do banco de dados com as seguintes tabelas
(em ordem de dependencia):

1. profiles (usuarios)
2. producer_profiles (dados de produtor)
3. events (eventos)
4. ticket_types (tipos de ingresso)
5. ticket_orders (pedidos)
6. ticket_order_items (itens do pedido)

Crie o arquivo SQL em `supabase/migrations/00000000000000_init_schema.sql`

Pergunte-me se tiver duvidas, mas tente ser autonomo.

5.2 Prompt para RLS Policies

PROMPT 2 - RLS Policies:

Agora crie as Row Level Security (RLS) policies para as tabelas que voce criou. Siga estas regras:

- Toda tabela deve ter RLS habilitado
- Usuarios autenticados so podem ver seus proprios dados
- Produtores so podem ver dados de seus proprios eventos
- Admins podem ver tudo
- Anonimos nao podem ler nada

Crie o arquivo em supabase/migrations/00000000000001_rls_policies.sql

Teste mentalmente cada policy: um usuario malicioso tentando acessar dados de outro usuario DEVE ser bloqueado.

5.3 Prompt para Substituir Mocks

PROMPT 3 - Substituir Mocks:

Substitua os dados mockados da pagina de listagem de eventos (src/pages/events/EventList.tsx ou equivalente) por dados reais do Supabase.

Requisitos:

1. Use o hook useEvents (crie se nao existir) com TanStack Query
2. Use o Zustand store de eventos (eventStore)
3. Implemente loading state com skeleton
4. Implemente tratamento de erro
5. Use paginacao (12 eventos por pagina)
6. Filtros: cidade, categoria, data, busca por texto

Nao remova o layout visual existente. Apenas substitua a fonte de dados de mock para API real. Execute npm run build e corrija erros antes de finalizar.

5.4 Prompt para Componentes de Pagamento

PROMPT 4 - Componentes de Pagamento:

Crie os componentes de pagamento seguindo o documento D10 (Frontend de Pagamentos). O gateway ainda NAO foi definido, entao todos os componentes devem ser gateway-agnostic.

Crie nesta ordem:

1. src/types/payments.ts - Todas as interfaces
2. src/components/payments/PaymentMethodSelector.tsx
3. src/components/payments/CreditCardForm.tsx (com validacao Luhn)
4. src/components/payments/PixPayment.tsx (QR code + timer)
5. src/components/payments/CheckoutSummary.tsx
6. src/pages/checkout/CheckoutScreen.tsx (integra tudo)

O adapter de pagamento (src/api/paymentAdapter.ts) deve ter apenas a interface, sem implementacao. A implementacao vem depois quando o gateway for escolhido.

5.5 Prompt para Edge Function

PROMPT 5 - Edge Function de Pagamento:

Crie a Edge Function supabase/functions/process-payment que:

1. Recebe um PaymentRequest via POST
2. Valida os dados (cartao, valor, metodo)
3. Verifica se o usuario esta autenticado
4. Salva a ordem no banco (tabela orders)
5. Retorna um PaymentResult com status "pending"
6. Usa o adapter pattern (a implementacao do gateway vem depois)

Nao implemente a chamada ao gateway ainda. Apenas crie a estrutura, validacao e persistencia no banco.

Teste localmente com: supabase functions serve

5.6 Prompt para Correcao de Erros

PROMPT 6 - Corrigir Erros de Build:

O build falhou. Execute "npm run build" e corrija TODOS os erros:

1. Erros de TypeScript (tipos incorretos, any implicito)
2. Erros de import (modulos nao encontrados)
3. Erros de ESLint (regras quebradas)
4. Warnings que possam causar problemas

Nao ignore nenhum erro. Cada erro deve ser corrigido na fonte.

Se precisar adicionar um tipo, adicione. Se precisar remover um import nao usado, remova.

Execute o build novamente ate passar sem erros.

5.7 Prompt para Testar Fluxo

PROMPT 7 - Testar Fluxo Completo:

Faca uma revisao completa do fluxo critico do aplicativo:

1. Usuario acessa pagina inicial
2. Ve listagem de eventos (dados reais do Supabase)
3. Clicka em um evento e ve detalhes
4. Seleciona tipo de ingresso e quantidade
5. Vai para checkout
6. Preenche dados (mock ou real)
7. Completa "pagamento" (ainda sem gateway real)
8. Ve tela de confirmacao

Identifique e corrija:

- Paginas que ainda usam mock
- Estados de loading ausentes
- Tratamento de erro ausente
- Links quebrados
- Layout quebrado em mobile

Crie um relatorio com tudo que encontrou e corrija o que for critico. O que for melhoria, liste como tarefa futura.

6. Workflow Diario com o Antigravity

6.1 Rotina Recomendada

Tabela 3 Workflow Diario de Desenvolvimento

Hora	Acao	Prompt/Oque fazer
Inicio	Atualizar contexto	"Hoje vamos trabalhar em [X]. O estado atual e [Y]."
Manha	Desenvolvimento	Use os prompts da Secao 5
Meio-dia	Build e teste	"Execute npm run build e corrija erros"
Tarde	Continuar desenvolvimento	Proxima tarefa da lista
Fim do dia	Review e commit	"Faca um resumo do que foi feito hoje"

6.2 Como Dar Feedback ao Antigravity

Quando o Antigravity gerar codigo, voce tem tres opcoes:

Tabela 4 Comandos de Feedback

Feedback	Como fazer	Quando usar
Aprovar	"Perfeito, aprove." ou "LGTM (Looks Good To Me)"	Codigo esta correto e completo
Modificar	"Mude [X] para [Y]" ou "Adicione [Z]"	Quase correto, precisa de ajustes
Rejeitar	"Nao, refaca. O correto e [explicacao]"	Codigo esta errado ou vai na direcao errada
Explicar	"Explique o que esse codigo faz"	Voce nao entendeu e quer aprender
Simplificar	"Simplifique esse codigo"	Muito complexo, pode ser mais simples
Testar	"Crie um teste para essa funcao"	Quer garantir que funciona

7. Troubleshooting (Problemas Comuns)

7.1 O Antigravity Nao Encontra os Arquivos

Problema: A IA diz que nao consegue encontrar um arquivo ou nao sabe onde algo esta.

Solucao: Sempre use caminhos absolutos ou relativos claros. Exemplo: "O arquivo esta em `src/pages/producer/Dashboard.tsx`, nao em `src/components/`".

7.2 O Antigravity Gera Codigo Fora do Padrao

Problema: A IA usa JavaScript puro em vez de TypeScript, ou cria CSS em vez de Tailwind, ou ignora as convenções.

Solucao: Lembre-a das regras: "Lembre-se: usamos APENAS TypeScript, Tailwind CSS e shadcn/ui. Nao crie arquivos .css. Nao use var."

7.3 O Antigravity Faz Mudancas em Arquivos que Nao Deveria

Problema: A IA modifica arquivos existentes (como App.tsx ou layout) sem permissao.

Solucao: Seja explicito: "Nao modifique App.tsx, Header.tsx ou qualquer layout. Apenas crie o novo componente e mostre como importa-lo."

7.4 O Build Quebra Apos Alteracoes

Problema: Depois de varias alteracoes, npm run build da erro.

Solucao: Use o Prompt 6 (Correcao de Erros). Se persistir, peca: "Faca um git diff e mostre o que mudou desde o ultimo build funcionando".

7.5 O Antigravity Esquece o Contexto em Conversas Longas

Problema: Em conversas muito longas, a IA pode "esquecer" decisoes anteriores.

Solucao: Reinicie o contexto periodicamente: "Resumo do projeto: estamos construindo o Aura (SaaS de eventos). Ja fizemos [X]. Agora vamos fazer [Y]. Lembre-se das regras em .antigravity/rules.md."

8. Checklist de Configuracao do Antigravity

Use este checklist para garantir que tudo esta configurado antes de comecar a desenvolver:

Tabela 5 Checklist de Configuracao Completa

#	Tarefa	Status
1	Instalar Antigravity e fazer login	[]
2	Importar projeto Aura (GitHub ou upload)	[]
3	Criar pasta <code>.antigravity/</code>	[]
4	Criar <code>.antigravity/rules.md</code> (copiar da Secao 2.2)	[]
5	Criar <code>.antigravity/context.md</code> (copiar da Secao 2.3)	[]
6	Criar <code>.antigravity/conventions.md</code> (copiar da Secao 4.1)	[]
7	Criar <code>.antigravity/security.md</code> (copiar da Secao 4.2)	[]
8	Fazer upload dos documentos D1 a D10 como contexto	[]
9	Testar: colar Prompt 1 e verificar se entendeu	[]
10	Verificar se o build inicial passa (npm run build)	[]

9. Resumo

Para que o Antigravity desenvolva o Aura de forma autonoma, voce precisa de quatro arquivos na pasta `.antigravity/` :

Tabela 6 Arquivos Essenciais do Antigravity

Arquivo	Funcao	Onde Copiar
<code>rules.md</code>	Regras de comportamento, stack, prioridades	Secao 2.2 deste documento
<code>context.md</code>	Estado atual, o que falta, proxima tarefa	Secao 2.3 deste documento
<code>conventions.md</code>	Padroes de codigo, nomenclatura, organizacao	Secao 4.1 deste documento
<code>security.md</code>	Regras de seguranca inquebraveis	Secao 4.2 deste documento

Alem disso, faca upload dos documentos D1 a D10 como contexto na IDE, e use os prompts prontos da Secao 5 para guiar o desenvolvimento. Siga o workflow da Secao 6 e consulte a Secao 7 quando houver problemas.

Com estas configuracoes, o Antigravity tera todo o contexto necessario para:

- Criar o banco de dados Supabase seguindo o schema do D5
- Implementar RLS policies seguras
- Criar stores, hooks e componentes seguindo os padroes definidos
- Substituir mocks por integracoes reais
- Criar componentes de pagamento gateway-agnostic
- Manter a seguranca e qualidade do codigo

10. Proximos Passos Imediatos

Apos configurar o Antigravity com este guia:

1. **Hoje:** Crie os 4 arquivos na pasta `.antigravity/` e faca upload do contexto
2. **Amanha:** Use o **Prompt 1** para iniciar o schema do Supabase
3. **Semana 1:** Finalize tabelas, RLS policies e triggers

- 4. **Semana 2:** Configure cliente Supabase, stores e hooks
- 5. **Semana 3:** Substitua mocks pelas primeiras integracoes reais
- 6. **Semana 4:** Componentes de pagamento + Edge Functions

O specdrive completo (D1 a D11) tem **159+ paginas** de especificacao tecnica. Com o Antigravity configurado corretamente, voce tera uma equipe de desenvolvimento de IA trabalhando 24/7 no Aura.